

SM-AI

2026 · 11_02

AWS

SageMaker AI

A practical tutorial on managed ML

TEAM 11_02

Arthur Bohin · Dalibor Švonavec · Ilias El Mariky ·

Francesco Barillari · Ralph Khairallah

PRESENTED

May 21, 2026 · 08:30

Of organizations now use
AI in at least one business
function.

88%

McKinsey Global AI Survey, 2026 — up from 78% in 2025, 55% in 2024.

Enterprises with at least one AI workload
in production

72%

Increased AI budgets in 2026 (median
+22% YoY)

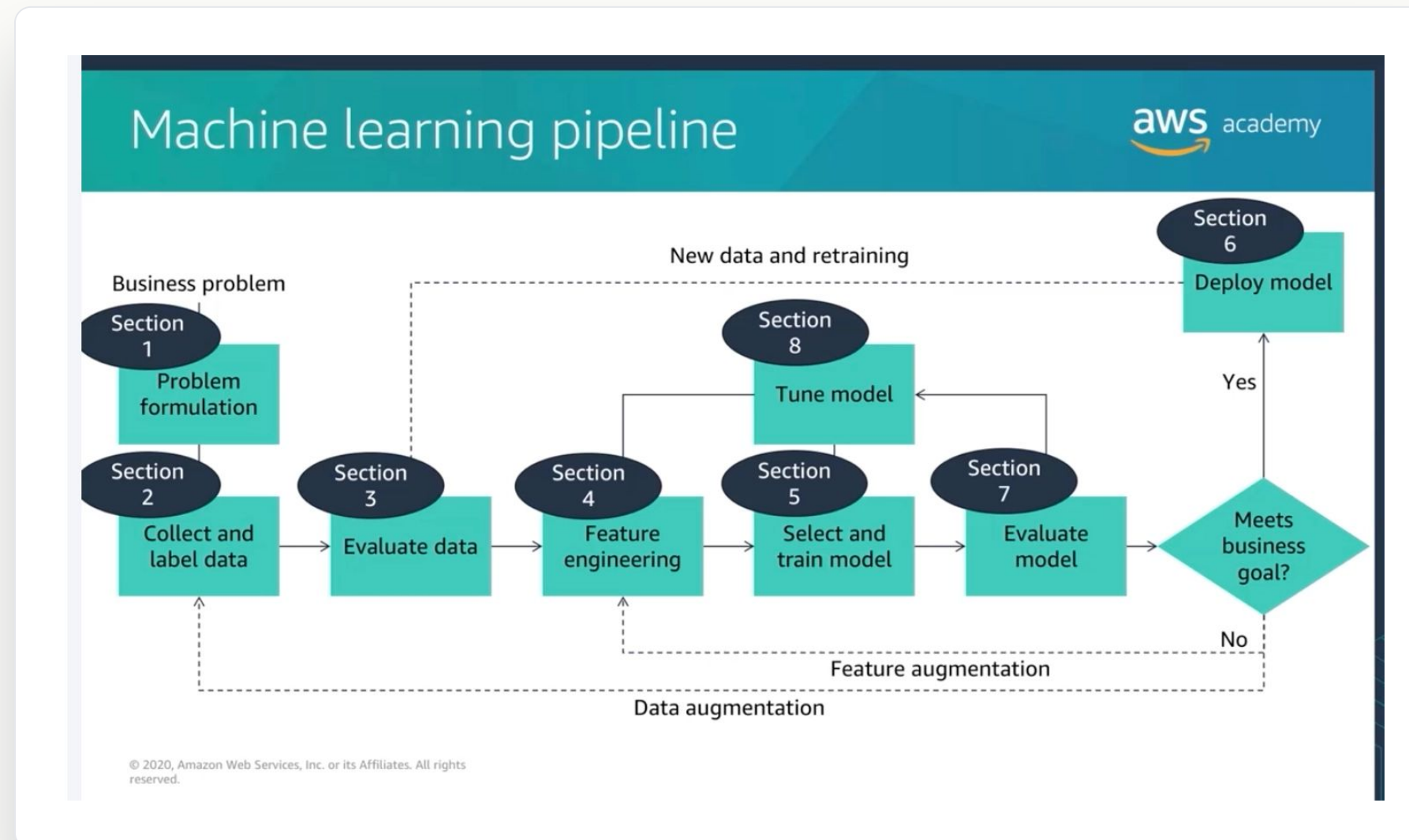
65%

Tech & software lead — financial
services 79%, healthcare 62%

88%

*“The era of AI as a competitive differentiator is
ending; AI is becoming the baseline.”*

— McKinsey Global AI Survey 2026



Seven stages. One platform.

SageMaker covers all 7 stages — from problem framing to monitored production. One service, one billing line, one set of APIs.

- | | |
|------------------------|----------------------------|
| 1. Problem formulation | 5. Model training |
| 2. Data collection | 6. Evaluation |
| 3. Cleaning | 7. Deployment + monitoring |
| 4. Feature engineering | ↳ all in SageMaker |

WHAT IS SAGEMAKER?

AWS's fully managed end-to-end ML platform — build, train, deploy, monitor , all from one place.

01 / LIFECYCLE

Covers the entire ML lifecycle — data prep, training, deployment, monitoring.

02 / BILLING

Pay-per-use. No GPU servers, no clusters, no infra to manage.

03 / INTEGRATION

Wired into S3, Lambda, IAM, CloudWatch — plays with the rest of AWS.

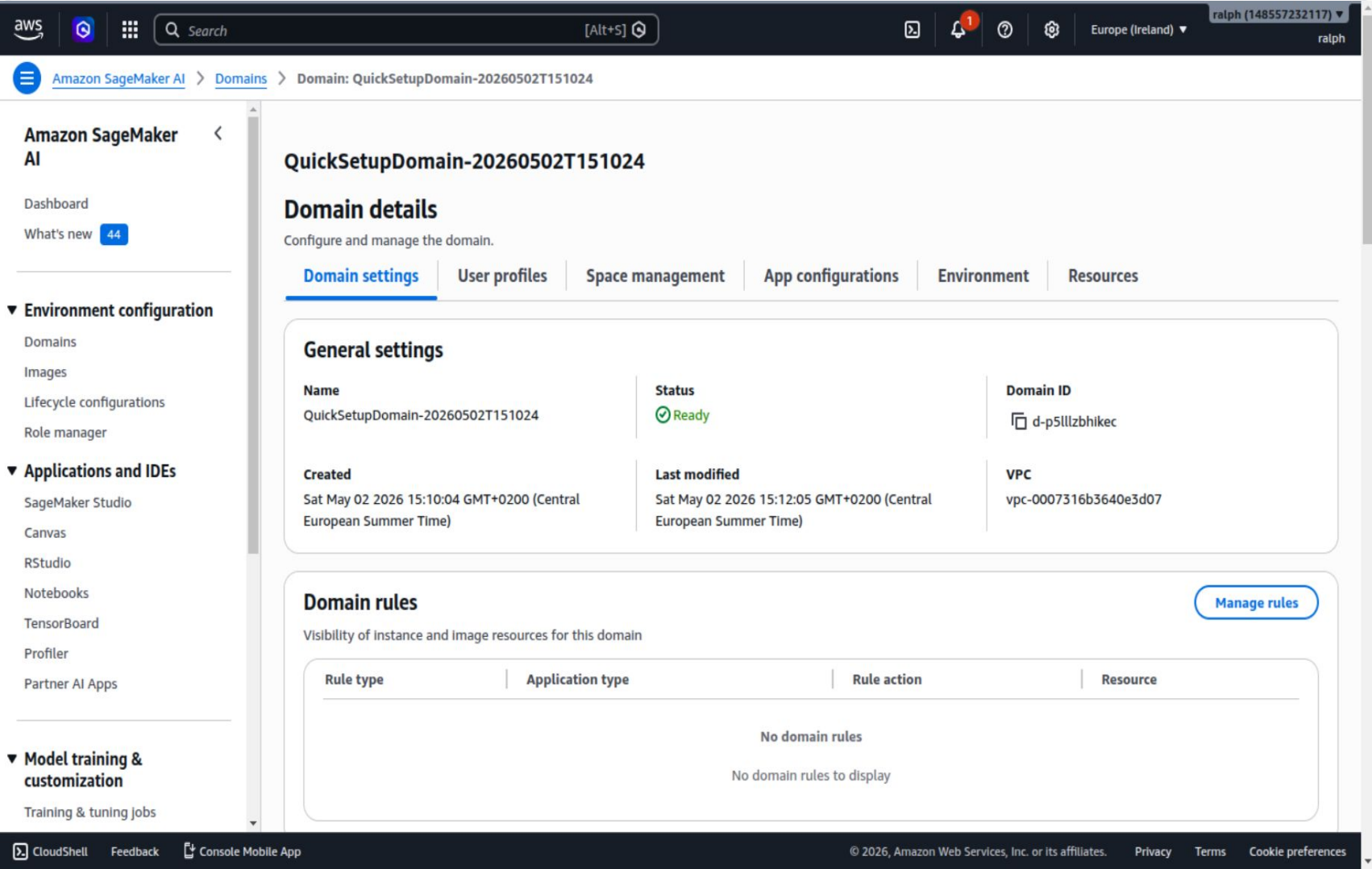
One service, many ways to use it.

SHAPE	WHAT IT IS	BEST FOR	INTERFACE
Canvas <i>no-code</i>	Visual ML builder, drag & drop	Business analysts, demos, quick prototypes	Web UI
Studio Notebooks <i>explore</i>	Cloud Jupyter, pre-loaded with ML libs	Data scientists exploring data	Jupyter
SDK / boto3 <i>production</i>	Python API to train, deploy, predict	Production engineers, automation, Lambdas	Python code
JumpStart <i>pre-trained</i>	Library of pre-trained models, 1-click deploy	Skipping training when a model already exists	Web UI + API
Pipelines <i>orchestrate</i>	Workflow orchestrator for ML steps	Production-grade reproducible ML	SDK + UI
Autopilot <i>autoML</i>	Auto algorithm selection, training, tuning	Teams that want best-model search without manual HPO	API + Canvas

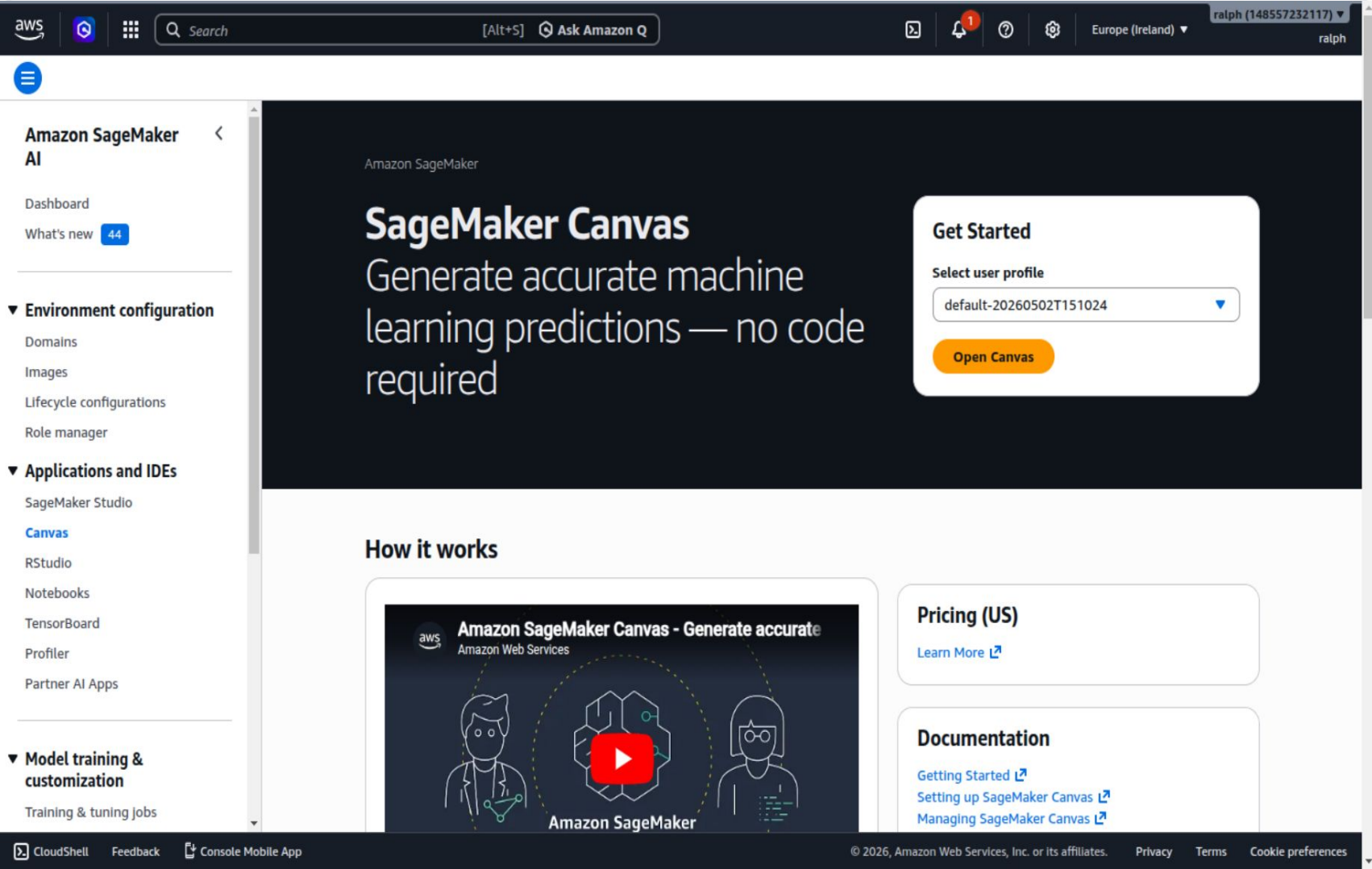
Same compute, same models, same endpoints — pick your interface based on your skill level.

Tutorial — from zero to a trained model.

STEP 1 · STEP 2 · STEP 3 · STEP 4

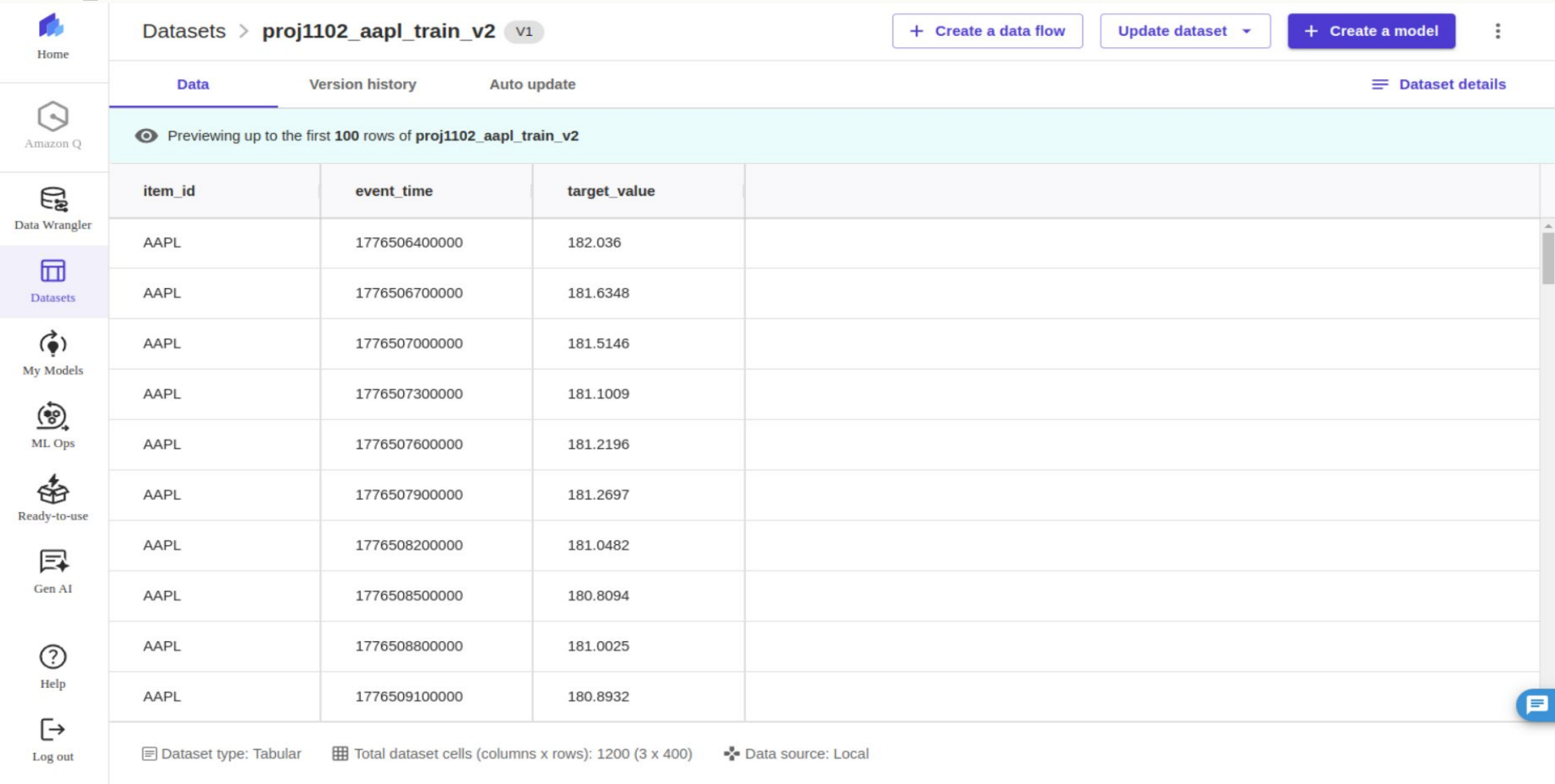


Studio domain — your shared ML workspace, ready in ~5 minutes.

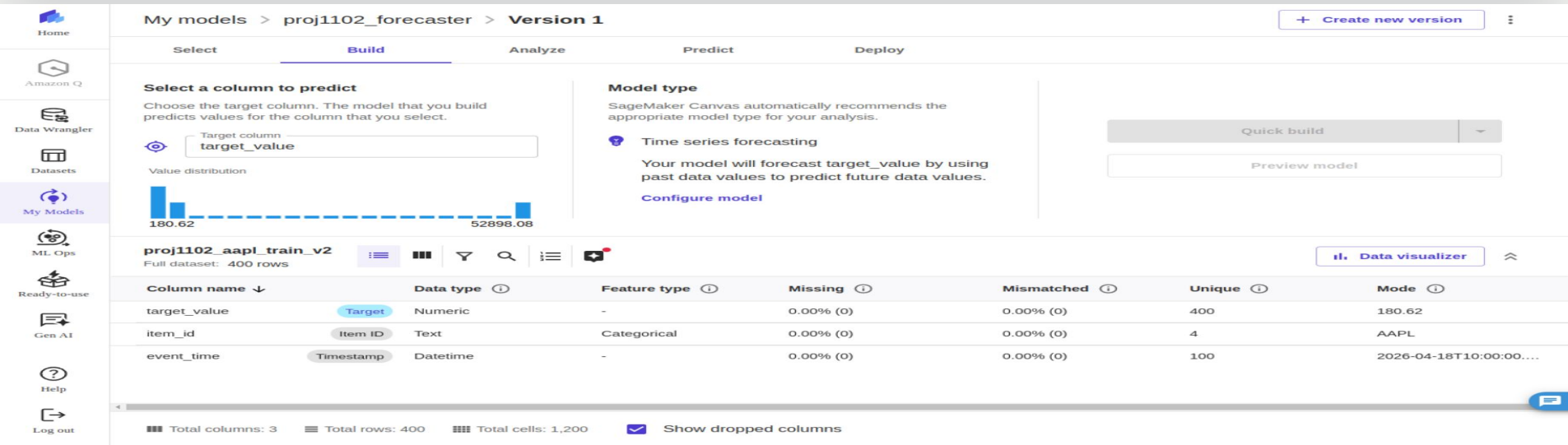


Canvas — the no-code entry point, one click away.

Upload data. Canvas does the rest.



Dataset preview — Canvas auto-detects column names & types.



Build tab — pick your target column. Done.

STEP 1 · **STEP 2** · STEP 3 · STEP 4

AUTO-DETECTED

item_id categorical

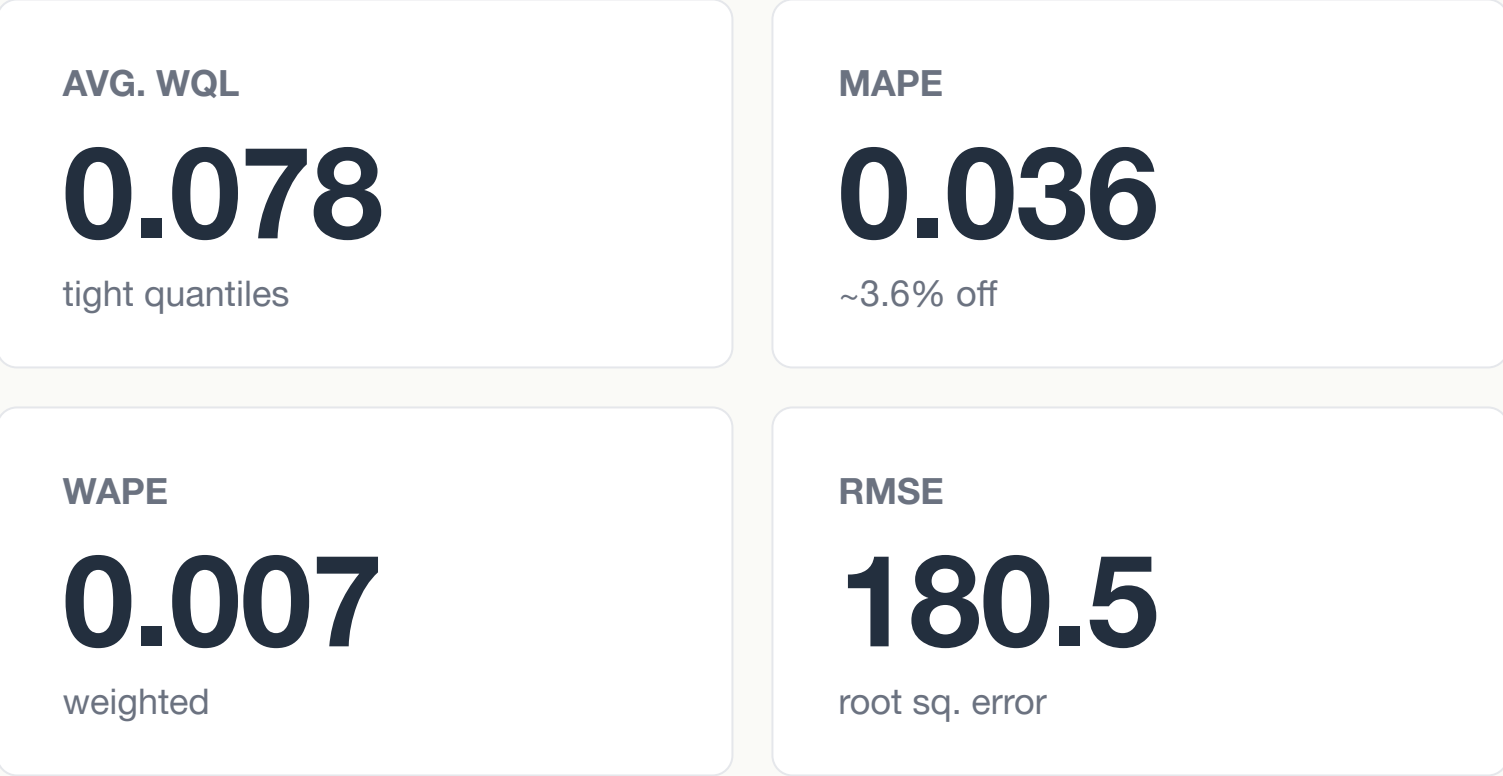
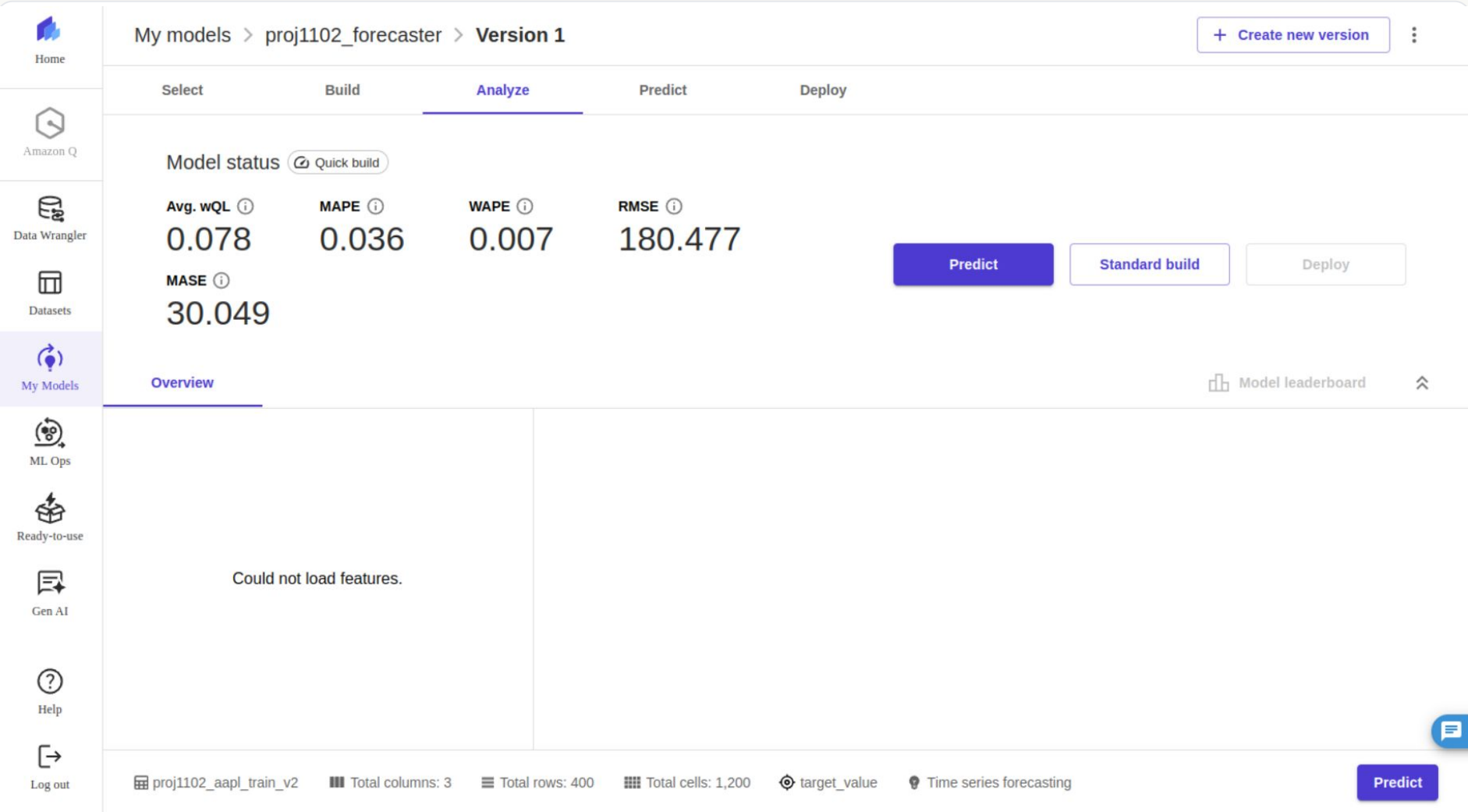
timestamp datetime

target_value numeric · target

Canvas inferred **time-series forecasting** from a CSV alone — no schema, no code.

No schema definitions. No type hints. Just a CSV.

One click. 14 minutes. A trained model.

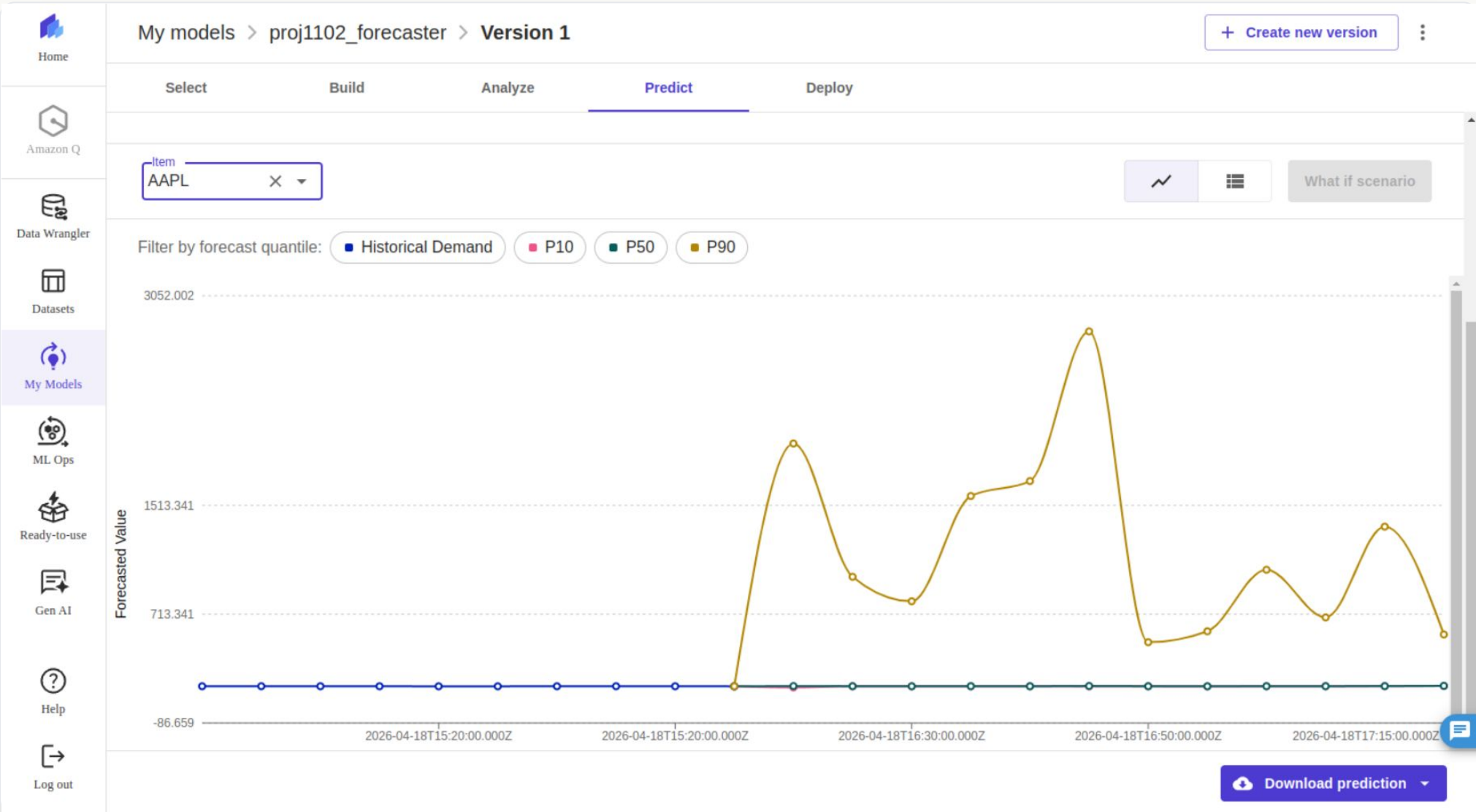


UNDER THE HOOD

DeepAR · ml.c5.xlarge · trained, validated and benchmarked — without provisioning a single server.

Forecasts with built-in confidence bands.

STEP 1 · STEP 2 · STEP 3 · STEP 4



WHAT YOU SEE

P50 hugs AAPL’s range correctly. **P90** spikes — we trained one model on dummy data mixing AAPL (\$180) with BTC (\$55,000) in one column.

Lesson. Canvas does not validate your data prep. Garbage in, garbage out — even on AWS.

Fix: normalize per item, or train one model per ticker. The platform is only as smart as the data you feed it.

Skip the UI. Drive SageMaker from **boto3**.

BOTO3 · PYTHON SDK

```
import sagemaker from sagemaker.image_uris import retrieve session =
sagemaker.Session() role = sagemaker.get_execution_role() image =
retrieve(framework="forecasting-deepar", region="eu-west-1") # Train
job - AWS-managed DeepAR container est = sagemaker.estimator.Estimator(
image_uri=image, role=role, instance_count=1,
instance_type="ml.c5.xlarge",
output_path="s3://my-bucket/deepar/output" , ) est.set_hyperparameters(
time_freq="5min", context_length=100, prediction_length=12,
epochs=50, num_cells=40, num_layers=2 , ) est.fit ({"train":
"s3://my-bucket/deepar/train.jsonl"}) # Deploy to a real-time endpoint
a Lambda can call predictor = est.deploy(instance_type="ml.t2.medium",
initial_instance_count=1) predictor.predict ({"instances": [{"start":
"...", "target": [...]}]}))
```

SAME WORKFLOW

Same DeepAR model. Same training. Same metrics — the UI just disappears.

- ↳ Re-run from a Lambda on a schedule
- ↳ Version it in git, plug into CI
- ↳ Swap algorithms with a one-line change
- ↳ Tune hyperparameters via SageMaker HPO

20+ built-in algorithms. Here are the ones you'll see most.

CATEGORY	ALGORITHMS	USE CASE
General purpose	XGBoost, Linear Learner, AutoGluon-Tabular, CatBoost	Classification & regression on tabular data
Time series	DeepAR	Forecasting (next slide)
Unsupervised	K-Means, PCA, Random Cut Forest	Clustering, dimensionality reduction, anomaly detection
Text / NLP	BlazingText, Seq2Seq, LDA	Word embeddings, translation, topic modeling
Vision	Image Classification, Object Detection, Semantic Segmentation	Image labeling, object detection, pixel-level tagging

NOT BUILT-IN — DIFFERENT SERVICES

LLMs / generative AI → **Bedrock**

Pre-trained models (Hugging Face, Stable Diffusion, etc.) → **JumpStart**

One built-in algorithm. One AutoML ensemble. Pick your level of control.

APPROACH	WHAT IT IS	BEST FOR
DeepAR our pick	RNN-based deep learning forecaster	Full control, many related series jointly (e.g. all 4 tickers + sentiment)
Autopilot ensemble minimal effort	Trains 6 models, stacks the best (ARIMA, Prophet, NPTS, ETS, CNN-QR, DeepAR+)	Best model with minimal effort
↳ ARIMA baseline	Classical statistical model	Simple datasets, < 100 series
↳ Prophet interpretable	Additive model, trend + seasonality	Single series, interpretable

DECISION QUICK-RULE

Multiple series, full control → DeepAR

One series, simple → Prophet / ARIMA via Autopilot

What it actually costs.

FREE TIER · FIRST 2 MONTHS

Studio Notebooks ml.t3.medium	250 h/mo
Training jobs ml.m5.xlarge	50 h/mo
Real-time inference ml.m5.xlarge	125 h/mo
Data Wrangler ml.m5.4xlarge	25 h/mo

Covers most learning use cases — our team has spent < \$5 so far.

⚠️ PROBLEM — ZOMBIE RESOURCES

Deleted endpoints can leave attached EBS volumes, elastic inference accelerators, or notebook resources billing forever.

AFTER FREE TIER · EU-WEST-1 · MAY 2026

Studio Notebook small	\$0.05 /h
Canvas session	\$1.13 /h
Training (CPU / GPU)	\$0.27 → \$4.20 /h
Real-time endpoint	\$0.06 – \$0.27 /h

A small idle endpoint on 24/7 ≈ \$200/mo even if no one calls it.

✓ SOLUTION — CLOUDWATCH BILLING ALARMS

Set a CloudWatch billing alarm before you start — cost monitoring is not automatic on SageMaker.

SageMaker is one of many options.

TOOL	CLOUD	BEST FOR
AWS SageMaker	AWS	Full ML lifecycle on AWS
Google Vertex AI	GCP	Same idea on GCP
Azure ML	Azure	Same idea on Azure
MLflow on K8s	Cloud-agnostic	Avoiding lock-in
COMPLEMENTARY AWS SERVICES		
AWS Bedrock	AWS	LLMs / generative AI
AWS Forecast	AWS	Deprecated for new accounts (2024)

DECISION RULES

- LLM / chatbot → **Bedrock**
- Time series, beginner → **Canvas**
- Time series, production → **SDK + DeepAR**
- Tabular ML → **SDK + XGBoost**
WHEN SAGEMAKER ISN'T THE FIT
- Avoiding lock-in → **MLflow on K8s**

Who actually uses it?

88% of organizations now use AI in at least one business function — McKinsey 2026

10,900+

Stars · GitHub

7,000+

Forks · GitHub

MLflow

Native integration

Figma

Perplexity

NFL

LG AI Research

Toyota

Salesforce

Itaú · banking

Cerner · healthcare

Intuit

Sophos

LOVED

- ✓ One-touch deployment
- ✓ Fully managed, scales easily
- ✓ Wide framework support — TF / PyTorch / MXNet
- ✓ Strong technical support

COMPLAINED ABOUT

- ✗ Opaque pricing → "month-end shock"
- ✗ Steep learning curve for non-AWS natives
- ✗ "Zombie resources" keep billing after deletion
- ✗ Vendor lock-in — SageMaker-specific artifacts

FLAGSHIP USE CASE

NFL Next Gen Stats scores every play of every game in real time using SageMaker — including live Completion Probability during broadcasts.

Lessons we learned **the hard way.**

01 Service quotas

Quotas hit zero by default for new accounts — especially GPU instances.

→ [File a quota request — plan for 24–48h delay](#)

02 Region matching

SageMaker, S3, QuickSight, DynamoDB — all in one region. Silent failures otherwise.

→ [Everything in eu-west-1, never mix](#)

03 Cross-account S3

IAM permissions on your side alone aren't enough. Add explicit bucket policy on S3 side.

→ [Add bucket policy on S3 side](#)

04 Idle billing ~\$200/month if endpoint left on 24/7

Canvas keeps billing while idle — always log out. Closing the tab does NOT stop the session.

→ [Set a CloudWatch billing alarm before you start](#)

05 Reserved keyword: timestamp

Reserved in Athena AND Canvas — both reject it silently. We lost 30 min on this.

→ [Rename to event_time](#)

06 AlgorithmError opaque

When a training job fails, the error message is useless.

→ [Always check CloudWatch container logs — enable debug from the start](#)

Our pivot. Where to dig deeper.

OUR PIVOT

AWS Forecast → SageMaker. Forecast was deprecated for new accounts — we hit the wall on day one of week two.

DeepAR is the spiritual successor to Forecast — same algorithm, lower level of abstraction.

PERSPECTIVE

SageMaker = comfort.
Open source = freedom.
The real question: are you ready
to bet on AWS for the next 10 years?

→ Unified Studio GA — March 2025: analytics + ML + GenAI unified

→ Challenger: MLflow on K8s, TrueFoundry — cloud-agnostic, zero lock-in

→ SageMaker Pipeline → full rewrite if migrating to another cloud

SageMaker Unified Studio · GA March 2025

RESOURCES · WHERE TO DIG DEEPER

Official docs

aws.amazon.com/sagemaker/ai/

Examples · 10.9K stars

github.com/aws/amazon-sagemaker-examples

Free Studio Lab — no AWS account

studiolab.sagemaker.aws

DeepAR paper

arxiv.org/abs/1704.04110